PC COMMAND AGENT

Go Migration & Production Architecture Plan

GOLANG | SINGLE EXE | API KEY AUTH | WINDOWS SERVICE | PRODUCTION

Document Type	Technical Architecture & Migration Plan
Target Language	Go (golang) 1.22+
Delivery	Single .exe — zero external dependencies
Security Model	PBKDF2 API Keys + HTTPS/TLS + Windows Service
Scope	Full feature parity with agent_v12.py + production hardening
Author	Ritik Saini
Date	17 May 2026
Version	v1.0 — Initial Migration Blueprint

This document provides a complete, step-by-step blueprint for migrating PC Command Agent from Python to Go — producing a single self-contained Windows executable with no runtime dependencies, production-grade security, and full feature parity with v12. It covers architecture, directory layout, all API endpoints, security hardening, Windows Service integration, build pipeline, and error-handling strategy.

Table of Contents

1. Why Go? — Motivation & Advantages

Python is excellent for rapid prototyping but has fundamental limitations for a production Windows agent: the GIL limits true parallelism, distributing requires bundling a runtime or using pyinstaller (large, slow, flagged by antivirus), and startup time is high. Go eliminates all of these.

Concern	Python (Current)	Go (Target)
Distribution	PyInstaller ~80MB, slow start	Single .exe ~8-12MB, instant start
Concurrency	GIL — fake threads	Real goroutines — millions concurrent
Startup time	2-5 seconds (imports)	< 50ms
Memory usage	80-150MB baseline	15-30MB baseline
Windows API	ctypes wrappers (fragile)	golang.org/x/sys/windows (first-class)
AV detection	PyInstaller triggers heuristics	Native binary — minimal false positives
Single binary	Complex (pyinstaller, spec files)	go build -o agent.exe (built-in)
Cross-compile	Hard	GOOS=windows GOARCH=amd64 go build
CGO required	Yes (many libs)	No (pure Go for all features needed)

2. Architecture Overview

The Go agent is a single binary containing two embedded HTTP servers (identical to the Python version), a Windows Service host, a system tray, and all Windows API calls via the golang.org/x/sys/windows package. No external DLLs, no Python, no ffmpeg dependency at runtime (audio encoding is done in pure Go using the minimp3 or oto library).

agent.exe (single binary)	
main.go	Entry point, Windows Service host, tray
server/api.go	Port 5000 — command API (Gin/Chi router)
server/stream.go	Port 5001 — MJPEG video + audio HTTP
input/mouse.go	SendInput mouse — relative & absolute
input/keyboard.go	SendInput keyboard — VK codes, combos
input/clipboard.go	Win32 clipboard read/write
capture/screen.go	mss-equivalent via BitBlt / DXGI
audio/wasapi.go	WASAPI loopback capture (pure Go)
audio/mp3.go	PCM → MP3 encode (beep/minimp3)
auth/keys.go	PBKDF2 key verify, rate limit, IP list
winsvc/service.go	Windows Service lifecycle (golang svc)
config/config.go	JSON config load/save with file lock
logger/logger.go	Rotating file log + stdout
embed/	Embedded tray icon, viewer HTML (go:embed)

3. Project Directory Layout

```
pc-agent/
cmd/agent/main.go ← entry point + service host
internal/
server/
api.go ← port 5000 all command endpoints
stream.go ← port 5001 MJPEG + audio + viewer
middleware.go ← auth, rate limit, logging, CORS
input/
mouse.go ← SendInput mouse (relative + absolute)
keyboard.go ← SendInput keyboard VK + combos
clipboard.go ← Win32 clipboard via windows.OpenClipboard
desktop.go ← SetThreadDesktop for UAC/login screen
capture/
screen.go ← BitBlt frame grabber (background goroutine)
dxgi.go ← DXGI Desktop Duplication (faster, optional)
audio/
wasapi.go ← WASAPI loopback via IMMDeviceEnumerator
encoder.go ← PCM → MP3 via minimp3/lame CGO or pure Go
auth/
auth.go ← PBKDF2 verify, plaintext guard, master key
ratelimit.go ← token-bucket per IP
iplist.go ← allowlist check
config/
config.go ← load/save agent_config.json
keygen.go ← PBKDF2 hash generation for setup
winsvc/
service.go ← golang.org/x/sys/windows/svc integration
logger/
logger.go ← rotating file log (lumberjack)
tray/
tray.go ← systray goroutine (getlantern/systray)
winapi/
winapi.go ← raw Windows API proc calls
embed/
viewer.html ← embedded browser viewer (go:embed)
icon.ico ← tray icon (go:embed)
go.mod ← module definition
go.sum ← dependency lock
build.bat ← GOOS=windows build script
installer.bat ← service install + firewall + key setup
```

4. Phase 1 — Core HTTP Server & Port Setup

PHASE 1 **Core HTTP Server & Port Setup**

- Use net/http standard library with custom ServeMux — no external framework needed for this use-case.
- Port 5000: create http.Server{Addr:':5000', Handler:apiMux, ReadTimeout:30s, WriteTimeout:60s}
- Port 5001: create http.Server{Addr:':5001', Handler:streamMux, WriteTimeout:0} — WriteTimeout=0 for streaming
- Run both servers in separate goroutines. Each has its own ServeMux so routes never collide.
- Socket tuning via net.ListenConfig + syscall.SetsockoptInt: SO_SNDBUF 16MB, SO_RCVBUF 16MB, TCP_NODELAY=1
- Graceful shutdown: os/signal SIGTERM → context.WithTimeout(10s) → both servers Shutdown(ctx)
- Keep-alive: http.Server.SetKeepAlivesEnabled(true), IdleTimeout: 120s
- Thread pool equivalent: Go's goroutine-per-request model handles thousands concurrent natively.
- go.sum lock file pins all dependency versions — reproducible builds guaranteed.

5. Phase 2 — Security: API Keys, TLS, Rate Limiting

PHASE 2 **Security: API Keys, TLS, Rate Limiting**

- ■ PBKDF2-HMAC-SHA256 key verification: crypto/pbkdf2.Key(password, salt, 260000, 32, sha256.New)
- ■ Keys stored ONLY as 'salt_hex:dk_hex' in agent_config.json — plain text keys never saved.
- ■ Middleware chain: auth.go → rateLimiter.go → ipAllowlist.go → handler
- ■ Rate limiter: token-bucket per remote IP using golang.org/x/time/rate — 60 req/min default.
- ■ IP allowlist: load []string from config; empty = allow all. Check net.IP.Equal on each request.
- ■ TLS: crypto/tls self-signed cert generated at first run via x509.CreateCertificate — saved to disk.
- ■ HTTPS on both ports 5000 and 5001. Cert generated with 2048-bit RSA + SHA256. Regenerates if expired.
- ■ Cert fingerprint shown in tray tooltip so user can pin it in the Android app.
- ■ Master key: separate PBKDF2 hash in config; only master key can change secret key or kick users.
- ■ No credentials ever logged. Log sanitizer strips X-Secret-Key header values from access logs.
- ■ Config file permissions: set to 0600 (owner-read-only) via os.Chmod after every write.
- ■ Startup config validation: if no keys configured, agent refuses to start and writes setup instructions.

6. Phase 3 — Windows Service + UAC/Login Screen Access

PHASE 3 **Windows Service + UAC/Login Screen Access**

- Import golang.org/x/sys/windows/svc — the official Go Windows service package.
- Implement svc.Handler interface: Execute(args, r, s) receives control signals (Stop, Shutdown).
- Install: sc create PCCommandAgent binPath= agent.exe start= auto obj= LocalSystem
- LocalSystem account gives SeTcbPrivilege needed to switch to the secure/interactive desktop.
- Desktop switching function: user32.OpenInputDesktop(0, false, DESKTOP_WRITEOBJECTS) then SetThreadDesktop.
- Call switchToInteractiveDesktop() at the START of every goroutine that calls SendInput.
- This is why UAC prompts and the login screen can now receive input — the agent's input thread attaches to the correct desktop.
- Service recovery: sc failure PCCommandAgent reset= 60 actions= restart/5000/restart/5000/restart/5000
- installer.bat wraps all service install steps: sc create, sc description, sc failure, sc start.
- Service status check: queryServiceStatus() called from tray tooltip every 30s.
- On service stop: both HTTP servers Shutdown(ctx), audio stream closed, tray icon removed.

7. Phase 4 — Screen Capture & MJPEG Video Stream

PHASE 4 **Screen Capture & MJPEG Video Stream**

- ■ Background goroutine: frame grabber using GDI BitBlt via windows.NewLazyDLL('gdi32.dll').
- ■ Grab loop: GetDC(0) → CreateCompatibleDC → CreateCompatibleBitmap → BitBlt → GetDIBits → JPEG encode.
- ■ JPEG encoding: github.com/disintegration/imaging or standard image/jpeg — pure Go, no CGO.
- ■ Shared frame slot: sync.RWMutex protecting []byte latestFrame. Grabber writes, stream handlers read.
- ■ Frame grabber sleeps when streamClients.Load() == 0 (atomic counter) — zero CPU when nobody watching.
- ■ Target FPS configurable: default 20fps. Sleep interval = (1/fps) - encodeTime.
- ■ DXGI Desktop Duplication (optional, faster): use IDXGIOutputDuplication via windows.IUnknown calls for GPU-accelerated capture.
- ■ MJPEG stream: multipart/x-mixed-replace boundary. Each part: Content-Type: image/jpeg + CRLF + JPEG bytes.
- ■ Stream endpoint reads from shared slot, applies quality/width override if non-default (same logic as Python).
- ■ Client count tracked with sync/atomic — increment on connect, decrement on GeneratorExit equivalent (ctx.Done()).
- ■ Browser viewer HTML embedded via //go:embed embed/viewer.html — served directly from binary.

8. Phase 5 — Audio (WASAPI Loopback)

PHASE 5 **Audio (WASAPI Loopback)**

- ■ WASAPI via COM: use windows.NewLazyDLL + CoCreateInstance for IMMDeviceEnumerator.
- ■ Enumerate endpoints: IMMDeviceEnumerator.EnumAudioEndpoints(eRender, DEVICE_STATE_ACTIVE).
- ■ Activate loopback client: device.Activate(IID_IAudioClient, CLSCTX_ALL, nil, &client;).
- ■ Initialize with AUDCLNT_STREAMFLAGS_LOOPBACK flag — this is the WASAPI loopback trick.
- ■ Capture loop: IAudioCaptureClient.GetBuffer → append to channelPCM → ReleaseBuffer.
- ■ PCM queue: buffered channel (chan []byte, 50) — same design as Python _audio_pcm_queue.
- ■ MP3 encoding option A (recommended): embed minimp3 encoder via CGO (tiny, ~70KB, LGPL).
- ■ MP3 encoding option B (pure Go): use github.com/hajimehoshi/oto + custom MP3 frame writer.
- ■ Audio toggle: sync.Mutex protected bool + sync.Cond to wake/sleep the capture goroutine.
- ■ Auto-reconnect: capture goroutine recovers from device disconnect using exponential back-off (100ms → 2s max).
- ■ Audio status endpoint: returns {enabled, streamingClients} from atomic counters — same API as Python.
- ■ Zero system audio changes: no volume, no device switching, no profile changes.

9. Phase 6 — Input Control (Mouse, Keyboard, SendInput)

PHASE 6 **Input Control (Mouse, Keyboard, SendInput)**

- ■ user32.SendInput via windows.NewProc('SendInput') — identical Win32 call as Python ctypes version.
- ■ MOUSEINPUT struct in Go: type MOUSEINPUT struct { Dx, Dy int32; Data, Flags, Time uint32; ExtraInfo uintptr }
- ■ KEYBDINPUT struct: type KEYBDINPUT struct { VkCode, Scan uint16; Flags, Time uint32; ExtraInfo uintptr }
- ■ INPUT union: type INPUT struct { Type uint32; _ [4]byte; Mi MOUSEINPUT } — padding handles union alignment.
- ■ VK map: var VK = map[string]uint16{'WIN':0x5B, 'CTRL':0x11, ...} — same as Python dict.
- ■ sendCombo: presses all keys in order, then releases in reverse — identical logic to Python _send_combo.
- ■ Mouse absolute: convert x,y to 0-65535 range: absX = x*65535/screenW — same formula as Python.
- ■ Keyboard type (Unicode): use KEYEVENTF_UNICODE flag with wScan=charCode for non-ASCII characters.
- ■ Clipboard type (fast path): OpenClipboard → EmptyClipboard → SetClipboardData(CF_UNICODETEXT) → CTRL+V.
- ■ Desktop switch: call switchDesktop() at start of EVERY sendInput call goroutine — critical for UAC screens.
- ■ Held keys tracking: sync.Mutex protected map[uint16]bool — release all on agent shutdown.
- ■ Drag support: atomic bool _dragActive + button type. Mouse down/up endpoints toggle it.

10. Phase 7 — File Operations & Browse Endpoints

PHASE 7 **File Operations & Browse Endpoints**

- ■ All file paths: filepath.FromSlash(path) converts forward slashes to backslashes — same as Python .replace.
- ■ File download: http.ServeContent for range-request support (resume support) — better than Python manual range.
- ■ File upload: io.Copy from r.Body to os.File with 4MB buffer — matches Python CHUNK_SIZE.
- ■ Chunked upload: same index/total logic; temp files named file.part0, file.part1 etc; reassemble on complete.
- ■ Browse directory: os.ReadDir + sort.Slice (dirs first, then files alphabetically) — matches Python scandir.
- ■ Drive enumeration: windows.GetLogicalDriveStrings + GetDiskFreeSpaceEx + GetVolumeInformation.
- ■ File search: filepath.Walk with deadline context (8 second timeout) — cancel via ctx.Done().
- ■ Special folders: os.UserHomeDir + filepath.Join for Desktop, Downloads, Documents, Pictures, Videos, Music.
- ■ App browser: check os.Stat for each KNOWN_APPS path; running check via windows.CreateToolhelp32Snapshot.
- ■ Recent files: os.ReadDir of %APPDATA%\Microsoft\Windows\Recent sorted by ModTime descending.
- ■ Permission errors: return 403 with {ok:false, error:'permission denied'} — consistent with Python.
- ■ Path traversal guard: filepath.Clean + strings.HasPrefix check before any file operation.

11. Phase 8 — System Commands & App Control

PHASE 8 **System Commands & App Control**

- Launch app: windows.ShellExecute(0, 'open', appPath, args, '', SW_SHOW) — same as Python ShellExecuteW.
- Kill process: windows.CreateToolhelp32Snapshot + Process32First/Next → find by name → OpenProcess → TerminateProcess.
- Window focus: EnumWindows callback finds HWND by PID → ShowWindow(SW_RESTORE) → SetForegroundWindow.
- Lock screen: user32.LockWorkStation() — same as Python ctypes call.
- Sleep: powrprof.SetSuspendState(false, false, false) via windows.NewProc.
- Shutdown/Restart: windows.InitiateSystemShutdownEx with reason code.
- Volume set: MMDevice API ISimpleAudioVolume.SetMasterVolume — same COM path as Python pycaw.
- Brightness: WMI query via windows.ConnectServer + ExecQuery for WmiMonitorBrightnessMethods.
- Screen wake: SetThreadExecutionState(ES_CONTINUOUS | ES_DISPLAY_REQUIRED) then SendInput SHIFT key.
- Unlock screen: switchToSecureDesktop() → simulate mouse click on password field → type via SendInput → ENTER.
- URL open: ShellExecute(0, 'open', url, '', '', SW_SHOW).
- Open folder in Explorer: exec.Command('explorer.exe', '/select,', path).Start()
- Task Manager: exec.Command('taskmgr.exe').Start() — no admin needed just to open it.

12. Phase 9 — Config, Logging, Watchdog & Connection Tracking

PHASE 9 **Config, Logging, Watchdog & Connection Tracking**

- Config: encoding/json + sync.RWMutex; os.Chmod(path, 0600) after every write.
- Log rotation: gopkg.in/natefinish/lumberjack.v2 — MaxSize:10MB, MaxBackups:5, Compress:true.
- Log format: time + level + message — same as Python logging.basicConfig.
- Connection tracking: sync.Map[deviceID]ConnInfo — concurrent safe, no explicit lock needed.
- Heartbeat goroutine: 30s ticker → SetThreadExecutionState, prune stale connections (>300s), log uptime.
- Request watchdog: each request runs in goroutine; context.WithTimeout(30s) cancels hung handlers.
- Stale connection pruner: range sync.Map, check lastSeen > 5min, delete and write disconnect log.
- Connection log files: one file per device_id in connection_logs/ — JSON lines, append-only.
- Health endpoint: /health returns uptime, version, audioEnabled, streamClients, connectedUsers.
- Job ID system for /execute: sync.Map[jobID]*Future; /execute/result/:id polls completion.
- Startup validation: if no keys in config → log.Fatal with setup instructions → os.Exit(1).
- Panic recovery middleware: recover() in each goroutine → log stack trace → return 500.

13. Phase 10 — Build Pipeline: Single .exe

Go produces a single statically-linked executable by default. With CGO_ENABLED=0 the binary has zero DLL dependencies. The entire viewer HTML, tray icon, and embedded assets are baked in via go:embed at compile time.

Step	Command / Action	Purpose
1	`go mod init github.com/yourname/pc-agent`	Initialize Go module
2	`go mod tidy`	Download & lock all dependencies
3	`go vet ./...`	Static analysis — catch bugs pre-build
4	`go test ./...`	Run all unit tests
5	`GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -ldflags='-s -w -H windowsgui' -o agent.exe ./cmd/agent`	Build release binary
6	`go tool nm agent.exe | grep -c ' T '`	Verify symbol count (sanity check)
7	`upx --best agent.exe  (optional)`	Compress binary ~60% smaller (~5MB)
8	`xcopy agent.exe installer\ /Y`	Copy to installer folder
9	`installer.bat`	Run setup: keys, firewall, service install

Key build flags explained:

`-ldflags='-s'`	Strip debug symbols (smaller binary)
`-ldflags='-w'`	Strip DWARF info (smaller binary)
`-H windowsgui`	No console window on launch (equivalent to pythonw.exe)
`CGO_ENABLED=0`	Pure Go — zero DLL dependencies
`GOOS=windows`	Cross-compile to Windows from any OS
`GOARCH=amd64`	64-bit target

14. Error Handling & Fallback Strategy

Go's explicit error returns force every failure path to be handled. The strategy below ensures the agent NEVER crashes silently — every error is logged, every subsystem has a fallback, and every HTTP handler returns a structured JSON error.

Subsystem	Error Scenario	Fallback Action
Screen Capture	BitBlt fails (driver issue)	Retry with pyautogui-equivalent GetDC; log warning
Screen Capture	No screen attached	Return last valid frame; log 'no display'
Audio WASAPI	Device disconnected	Exponential back-off retry (100ms→2s); log device name
Audio WASAPI	Library unavailable	Return PCM silence frames; stream continues
Audio MP3	Encoder error	Fall back to raw PCM stream; set Content-Type audio/L16
HTTP Server	Port in use	Log error + os.Exit(1) with message to check port
HTTP Server	Panic in handler	recover() middleware → 500 response + stack trace log
Config Load	File missing	Create default config skeleton; refuse start if no keys
Config Load	JSON corrupt	Log error + os.Exit(1) with repair instructions
SendInput	Secure desktop closed	Re-call OpenInputDesktop; retry input after 100ms
Windows Service	Service crash	SCM auto-restart (configured in installer.bat)
TLS Cert	Cert expired / missing	Auto-regenerate on startup; log new fingerprint
File Upload	Disk full	Return 507 Insufficient Storage with bytes_free field
App Launch	Exe not found	Return 404 with suggested paths to check
Auth	Invalid key	Return 401; increment fail counter; block IP after 10 fails

15. Security Model Summary

Transport

- ✓ TLS 1.2+ on both ports 5000 and 5001 (self-signed cert, auto-generated)
- ✓ Cert fingerprint shown in tray — pin it in the Android client
- ✓ HTTP Strict-Transport-Security header on all responses

Authentication

- ✓ Every endpoint (except /ping, /health) requires X-Secret-Key header or ?key= param
- ✓ PBKDF2-HMAC-SHA256 with 260,000 iterations — brute-force infeasible
- ✓ Salt stored with hash (salt_hex:dk_hex) — rainbow tables useless
- ✓ Plain-text keys NEVER stored in config or logged anywhere
- ✓ Master key separate from secret key — key change requires master key

Rate Limiting

- ✓ 60 requests/minute per IP (token bucket) — blocks automated scanning
- ✓ 10 consecutive auth failures → IP blocked for 15 minutes
- ✓ Streaming endpoints exempt from rate limit (they hold long connections)

Network

- ✓ Optional IP allowlist: comma-separated in config; empty = LAN-only recommendation
- ✓ Ports 5000 and 5001 — opened only via installer.bat (Windows Firewall rule)
- ✓ Bind to 0.0.0.0 — firewall is the perimeter; LAN use is the intended scope

Binary

- ✓ Build flags -s -w strip all debug symbols and DWARF info from the binary
- ✓ CGO_ENABLED=0 means no external DLL dependencies — smaller attack surface
- ✓ go:embed bakes all assets in — no loose files to tamper with at runtime
- ✓ Config file set to 0600 permissions — other users on the machine cannot read keys

Operational

- ✓ Windows Service runs as LocalSystem — required for desktop switching; accept this trade-off
- ✓ All sensitive values (keys) redacted in logs — log sanitizer strips header values
- ✓ Connection log per device in connection_logs/ — audit trail of who connected
- ✓ Master key never shown in tray info dialog (only first 4 chars shown)

16. Migration Execution Checklist

Follow this checklist in order. Each item should be verified before proceeding to the next. Estimated total migration time: 40-60 hours for an experienced Go developer.

SETUP

- [ ] Install Go 1.22+ from go.dev/dl
- [ ] go mod init github.com/yourname/pc-agent
- [ ] Create directory structure as per Section 3
- [ ] Add go:embed directives for viewer.html and icon.ico

CORE

- [ ] Implement dual HTTP server (port 5000 + 5001) in main.go
- [ ] Write middleware.go: auth check, rate limit, panic recovery, request logging
- [ ] Write config.go: load/save with RWMutex and 0600 chmod
- [ ] Write logger.go: lumberjack rotating file + stdout
- [ ] Write auth.go: PBKDF2 verify, master key check, IP allowlist
- [ ] Write keygen.go: CLI sub-command to generate hashed keys interactively

INPUT

- [ ] Write winapi.go: SendInput structs and call wrappers
- [ ] Write keyboard.go: VK map, sendKey, sendCombo, holdKey, releaseKey
- [ ] Write mouse.go: relative move, absolute move, click, scroll, drag
- [ ] Write clipboard.go: OpenClipboard, SetClipboardData, GetClipboardData
- [ ] Write desktop.go: OpenInputDesktop, SetThreadDesktop — call before every SendInput
- [ ] Port all key combos from Python COMBOS dict to Go map

CAPTURE

- [ ] Write screen.go: BitBlt frame grabber goroutine with shared RWMutex slot
- [ ] Implement JPEG encode with quality and width parameters
- [ ] Write stream.go: MJPEG multipart/x-mixed-replace generator
- [ ] Implement streamClients atomic counter — sleep grabber when 0 clients
- [ ] Embed viewer.html and serve from /screen/viewer endpoint

AUDIO

- [ ] Write wasapi.go: COM init, IMMDeviceEnumerator, IAudioClient loopback
- [ ] Write encoder.go: PCM → MP3 (minimp3 CGO or pure Go fallback)
- [ ] Implement audio toggle (POST /audio/toggle) with sync.Cond
- [ ] Implement /audio/status endpoint with atomic client counter
- [ ] Test audio stream in browser with MP3 and PCM fmt options

SYSTEM

- [ ] Port all SYSTEM_CMD handlers: lock, sleep, shutdown, restart, brightness, volume
- [ ] Port LAUNCH_APP: ShellExecute + window focus after launch
- [ ] Port KILL_APP: Toolhelp32 snapshot + TerminateProcess

- [ ] Port all browse endpoints: drives, dir, search, special, apps, recent
- [ ] Port all file operation endpoints: download, upload, chunk-upload, delete, move, copy, rename

SERVICE

- [ ] Write service.go using golang.org/x/sys/windows/svc
- [ ] Write installer.bat: sc create, sc failure, firewall rules, key setup
- [ ] Test service install/start/stop/uninstall cycle
- [ ] Verify UAC dialog interaction works after desktop switching
- [ ] Verify login screen interaction works (test by locking PC)

SECURITY

- [ ] Add TLS: x509 self-signed cert generation on first run
- [ ] Verify HTTPS on both ports in browser and curl
- [ ] Test rate limiter: send 70 req/min and verify 429 response
- [ ] Test IP block after 10 auth failures
- [ ] Verify plain keys never appear in log files
- [ ] Verify config file is 0600 after write

BUILD

- [ ] Write build.bat with all -ldflags and CGO_ENABLED=0
- [ ] Build and verify single agent.exe < 15MB
- [ ] Test on clean Windows VM with no Go installed — must run standalone
- [ ] Run go vet ./... and fix all warnings
- [ ] Run go test ./... — all tests pass
- [ ] Optionally: run UPX on binary to reduce size further

VALIDATION

- [ ] Test all endpoints against Python v12 reference implementation
- [ ] Test stream: 1080p, 2K, 720p quality levels in browser
- [ ] Test audio toggle, audio stream MP3 and PCM
- [ ] Test all keyboard combos from the Android app
- [ ] Test file upload and download with large files (>100MB)
- [ ] Test Windows Service auto-start after reboot
- [ ] Test UAC dialog interaction and login screen unlock
- [ ] Load test: 10 concurrent connections, verify no goroutine leak

End of Plan

PC Command Agent — Go Migration Plan | Prepared for Ritik Saini | May 2026